Lazy Enumerators

A lazy enumerator is formed by calling the method Enumerable#lazy. Usually it's the beginning of a chain of enumerator methods, as shown in the produce example earlier. Once lazy is part of the method chain, it changes the behavior of the entire chain so that values passed through the chain one at a time, rather than all at once.

In other words, if I have a chain of calls something like, x.map(&:foo).select(&:bar).map(&:baz).take(5), the entire array is mapped, then the entire array is filtered, then the entire array is mapped, and then the first five elements are retrieved. If the original object x is very long, or indeed if it is infinite, this chain can be quite slow, especially because we only want five elements. By making it lazy, as in x.lazy.map(&:foo).select(&:bar).map(&:baz).take(5), the behavior changes. The first element of x is mapped, then filtered through select, and if it passes, mapped through baz and added to the list for take. This continues one by one only until the take(5) is completed, meaning that if x is long, far fewer elements have been processed.

You can un-lazy an enumerator chain by calling eager, which allows you to convert the lazy enumerator to something that can be returned from a method or used as an argument to a method that expects an enumerator.

Hash

The Hash, which associates arbitrary indexes to arbitrary values, is the most flexible basic class in Ruby. Although Hash does implement each and Enumerable, hashes behave slightly differently than Arrays and Sets.

Creating Hashes

Hashes have a literal syntax that uses curly braces to associate keys with values. The original form of separating keys from values uses the => symbol, often called a *hash rocket*:

```
hash = {"a" => 1, "b" => 2, "c" => 3}
```

In general, you want the hash keys to be immutable values, and symbols are commonly used. If the key is a symbol, then you can use a colon to separate the key from a value. The colon will also convert strings to symbols. The keys in this literal are :a, :b, and :c, all symbols:

```
hash = {a: 1, "b": 2, c: 3}
```

You can use both styles in a single hash literal, but if you use the colon style, the key must be a string or a bare word. Anything else will result in a syntax error.

If the key is a symbol, and the symbol has a meaning in the current context, then you can use just the key and the value will be assumed to be the local value. So, the following two hashes are equivalent:

```
a = 1
b = 2
c = 3
{a: a, b: b, c: c}
{a:, b:, c:}
```

The class Hash also responds to [] to create new hashes, a rarely seen alternative to the literal syntax, Hash[a: 1, b: 2]. The argument can be key/value pairs, another hash, a list of two-ele-